

Start coding or [generate](#) with AI.

```
import pandas as pd
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize the features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert to PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)
```

```
import pandas as pd
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize the features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert to PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert to PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

# Define Softmax Regression model
class SoftmaxRegression(nn.Module):

    def __init__(self):
        super().__init__()

        # 4 input features -> 3 output classes
        self.linear = nn.Linear(4, 3)

    def forward(self, x):
        return self.linear(x)

# Create model
model = SoftmaxRegression()

# Define loss function
criterion = nn.CrossEntropyLoss()

# Define optimizer
optimizer = optim.SGD(model.parameters(), lr=0.01)

print(model)
# Number of training epochs
epochs = 100

for epoch in range(epochs):

    # Forward pass
    outputs = model(X_train)

    # Calculate loss
    loss = criterion(outputs, y_train)

    # Clear previous gradients
    optimizer.zero_grad()

    # Backpropagation
    loss.backward()

    # Update model parameters
    optimizer.step()

    # Display loss every 10 epochs
    if (epoch + 1) % 10 == 0:
        print("Epoch [{}/{}], Loss: {:.4f}".format(
            epoch + 1, epochs, loss.item()
        ))

SoftmaxRegression(
  (linear): Linear(in_features=4, out_features=3, bias=True)
)
Epoch [10/100], Loss: 1.0558
Epoch [20/100], Loss: 0.9707

```

```
Epoch [30/100], Loss: 0.8989
Epoch [40/100], Loss: 0.8386
Epoch [50/100], Loss: 0.7878
Epoch [60/100], Loss: 0.7448
Epoch [70/100], Loss: 0.7083
Epoch [80/100], Loss: 0.6770
Epoch [90/100], Loss: 0.6500
Epoch [100/100], Loss: 0.6265
```

```
epochs = 100

for epoch in range(epochs):

    outputs = model(X_train)

    loss = criterion(outputs, y_train)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    if (epoch + 1) % 10 == 0:
        print("Epoch [{}/{}], Loss: {:.4f}".format(
            epoch + 1, epochs, loss.item()
        ))
```

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix

# Evaluate the model
with torch.no_grad():

    # Predict test data
    outputs = model(X_test)

    # Get the class with highest score
    _, predicted = torch.max(outputs, 1)

# Convert tensors to arrays
y_pred = predicted.numpy()
y_true = y_test.numpy()

# Calculate metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)

print("\nConfusion Matrix:")
print(cm)
import matplotlib.pyplot as plt
import seaborn as sns

# Plot Confusion Matrix
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix")
plt.show()
```

```
# Plot Actual vs Predicted Classes
plt.figure(figsize=(8, 5))

plt.plot(y_true, 'o-', label='Actual')
plt.plot(y_pred, 'x--', label='Predicted')

plt.xlabel("Test Sample")
plt.ylabel("Class")
plt.title("Actual vs Predicted Classes")
plt.legend()
plt.show()
```

Accuracy : 0.8
Precision: 0.7944444444444445
Recall : 0.8
F1-Score : 0.7919191919191919

Confusion Matrix:

```
[[10  0  0]
 [ 2  6  2]
 [ 0  2  8]]
```



